

منصة الموظف الرقمي

Digital Employee Platform

مرجع شامل للمشروع — تحضيراً لمناقشة التخرج

الـ (Entities) · الـ Controllers · الـ Services · عقود تكامل الـ AI · سيناريوهات الـ Business Logic · أسئلة متوقعة في المناقشة
المحتويات: نظرة عامة · الـ Architecture · قاعدة البيانات (كل)

الجزء ١ — نظرة عامة على المشروع

١.١ إيه هو المشروع

متكامل لخدمة عملاء ذكية للمطاعم/الأعمال، يغطي التفاعل مع العميل عن طريق الـ backend هو نظام Digital Employee Platform النظام مبني حوالين فكرة الـ admin والـ owner تحليلي للـ dashboard وبيوفر، tickets والـ orders وبيدير الـ voice والـ chat ما بياناتهم تتداخل business يعني نفس النظام بيخدم أكثر من BusinessId كل حاجة في قاعدة البيانات مرتبطة بـ multi-tenancy مستقل عن الثاني من غير.

١.٢ المكونات الرئيسية

- والداتا business logic الرئيسي، بيدير كل الـ Backend الـ — NET 9 Web API.
- Nour Voice (اسمه في المشروع) Voice AI Agent و Chat AI Service: فيه جزئين — (FastAPI) منفصل AI Engine Agent API).
- EF Core Code-First قاعدة البيانات، عن طريق — SQL Server (DigitalEmployeeDB).
- Owner/Admin Dashboard تطبيق الـ — Frontend (React + Vite).
- business مدمج في موقع الـ — Customer Chat Widget.

١.٣ المستخدمين (User Roles)

- voice call أو الـ widget بيتفاعل من غير تسجيل دخول عن طريق الـ JWT/مش له حساب — Customer.
- (escalations) المتصعدة tickets موظف بيرد على الـ — User (Agent).
- والتقارير dashboard بيدير المنيو والإعدادات ويشوف الـ business صاحب الـ — Owner.
- وإحصائيات النظام كله businesses أعلى صلاحية، بيشفوف كل الـ — Admin.

١.٤ إيه اللي المشروع بيحله

هو اللي بيحول كل ده لبيانات منظمة (orders, tickets, analytics) وبيدي الفريق البشري queue واضح للحالات اللي محتاجة تدخلهم بس. AI بيتكفل بالرد التلقائي على الأسئلة الشائعة، وإنشاء الـ orders، ورصد أي مشكلة محتاجة تدخل بشري (escalation) — والـ Backend بدل ما الـ business يوظف موظفين يردوا على كل استفسار أو شكوى، الـ

٢.١ الطبقات الأربعة (Clean Architecture)

- Presentation Layer — ال Controller ال 29، JWT Middleware، Error Middleware، عليها الدخول؛ نقطة الدخول؛
- Service Layer — ال Services ال ال DTOs ال ال FluentValidation Validators ال ال business logic؛ كل ال
- Domain Layer — ال Entities ال ال 23، Enums كيان) ال ال Repository Interfaces، من غير أي dependency؛
- خارجية
- Data Access Layer (DAL) — ال EF Core Repositories، Unit of Work، ApplicationDbContext، Migrations ال

٢.٢ ال Design Patterns

- CRUD بدل تكرار ال ال <T> IRepository عزل الوصول للداتا جوه — ال Repository Pattern
- atomic واحدة transaction المشتركة في عملية واحدة تتحفظ في ال Repositories يضمن إن كل ال — ال Unit of Work
- مخصصة DTOs ليها entity مباشرة، كل ال Domain Entities مايشوفش ال ال client ال — ال DTO Pattern
- ال Controller يوصل لل request بيشتغل تلقائيا قبل ما ال ال Validator ليها DTO كل — ال FluentValidation

٢.٣ ال Authentication

(role). فيه كمان Google OAuth 2.0 كطريقة تسجيل دخول بديلة، بيتحقق من ال ID token عن طريق ال ال Infrastructure Layer. BusinessId، وuserId، وال policies بتتحقق من ال ال AdminOnly، OwnerOrAdmin، AgentOrOwnerOrAdmin (JWT هو شكل التوكن، و Bearer هي طريقة إرساله في ال ال Authorization header). التوكن بيحتوي على claims زي role، email، النظام بيستخدم JWT Bearer Token (مش JWT أو Bearer كحاجتين منفصلتين —

٢.٤ العلاقة بين ال Backend وال AI Engine

— هو اللي بيقرر ينشئ Order أو Ticket بناء على ال ال flags اللي ال ال AI بترجعها، ال ال AI نفسه مايكتبش حاجة في قاعدة البيانات مباشرة. الاتصال بينهم عن طريق REST calls بعقود محددة (هتفصل في الجزء ٧). ال ال Backend هو المسؤول عن كل قرار (orchestrator) ال ال AI Engine سيرفر (Python (FastAPI منفصل تماما عن ال ال .NET Backend.

الجزء ٣ — قاعدة البيانات: الـ Domain Entities

كيان أساسي في Domain layer/Models، كلهم (ما عدا استثناءات قليلة) مرتبطين بـ BusinessId عشان يضمنوا الـ multi-tenancy.

٢٣

Entity	الوصف والعلاقات
Business	شبه كلها مرتبطة بيه (1-to-Many). فيه بروفايل كامل: الاسم، النوع، العنوان، السوشيال ميديا، ساعات العمل، طرق الدفع المقبولة. الكيان المركزي — كل باقي الكيانات
User	من (ASP.NET Identity) IdentityUser. عنده Role: Admin/Owner/Agent/User، ومرتبطة بـ Business واحدة. بيمتد
Customer	(مختلف عن User اللي هو موظف). بيربط الـ Orders والـ Tickets والـ Interactions والـ Feedbacks بتاعته. عميل الـ business
Interaction	Open → InProgress → Escalated/Closed/Interrupted. بتربط بالـ RelatedTicketId أو RelatedOrderId لو نتج عنها حاجة. أي محادثة (Voice أو Channel: WebChat). ليها Status بيتنقل: (Voice أو Channel: WebChat).
Message	Interaction. SenderType: Customer/Agent/AI. ليها Sentiment مرتبط 1:1، ويمكن يكون معاها AudioPath لو صوتية. رسالة واحدة جوه
Sentiment	تحليل مشاعر رسالة واحدة: Score رقمي + Label (Positive/Neutral/Negative).
Order	عمل. Canceled/DelayedRisk، أو Status: Pending → PendingConfirmation → Paid → Delivered. أورد
OrderItem	عنصر واحد جوه الأورد — بيربط Order بـ MenuItem بكمية (Quantity) وسعر وقت الطلب (UnitPrice).
MenuItem	صنف في المنيو (اسم، وصف، سعر، IsAvailable). مرتبط بـ MenuCategory اختياريًا.
MenuCategory	تصنيف المنيو (بيتز، مشروبات...) مع DisplayOrder لترتيب العرض.
Ticket	.TicketType: LateDelivery/WrongOrder/MissingItem/PaymentIssue/QualityIssue/HumanEscalation. تذكرة دعم/شكوى. PriorityLevel: Low→Critical.
Feedback	تقييم من العميل (Rating من 1 لـ 5 + Comment)، ممكن يترابط بـ Ticket أو Interaction اختياريًا.
KnowledgeBase	قاعدة المعرفة (IsFAQ). Question/Answer. بيفرق بين سؤال شائع ظاهر للعميل ومعرفة داخلية للـ AI بس.
Notification	إشعار داخل النظام لليوزر أو للـ business ككل، مع IsRead flag.
AuditLog	سجل تدقيق شامل — بيسجل Action و Entity و EntityId و UserId لأي حركة مهمة حصلت في النظام.
Setting	إعدادات الـ business (1:1): تفعيل الشات بوت، رسالة الترحيب، شخصية الرد، مزود ولغة صوت الـ agent.
WorkingHours	ساعات العمل لكل يوم في الأسبوع (DayOfWeek 0-6، OpenTime/CloseTime، IsClosed).
CallSummary	مكالمة صوتية كاملة — الـ transcript، الـ sentiment، الـ topics، روابط الصوت — بيبجي من الـ AI بعد ما المكالمة تخلص. ملخص
InteractionAnalysis	بس للـ chat sessions — تحليل ما بعد المحادثة (summary, sentiment, intents, key moments). نفس فكرة CallSummary
Report	تقرير محفوظ (ReportType: Performance/Finance/ChatSummary) مع ملف مرفق قابل للتحميل.
Integration	تكاملات خارجية للـ business (Status: Active/Disabled).
Subscription	اشتراك الـ business في خطة معينة على المنصة.
PaymentTransaction	سجل عمليات الدفع المرتبطة بالـ (Subscription PaymentMethod: Card/Cash/Online).

الجزء ٤ — ال Enums وقيم الحالة الجاهزة

٤.١ ال Enums الرسمية

Enum	القيم
OrderStatus	Pending, PendingConfirmation, Paid, DelayedRisk, Delivered, Cancelled
PaymentStatus	Completed, Failed, Pending, Success
PaymentMethod	Card, Cash, Online
IntegrationStatus	Active, Disabled
ReportType	Performance, Finance, ChatSummary

٤.٢ قيم Status نصية (string-based، مش enum رسمي)

الحقل	القيم المحتملة
Interaction.Channel	WebChat, Voice
Interaction.Status	Open, InProgress, Escalated, Closed, Interrupted
Message.SenderType	Customer, Agent, AI
Ticket.Status	Open, InProgress, Escalated, Closed
Ticket.TicketType	LateDelivery, WrongOrder, MissingItem, PaymentIssue, QualityIssue, HumanEscalation
Ticket.PriorityLevel	Low, Normal, High, Critical
Sentiment.Label	Positive, Neutral, Negative
User.Role (Roles.cs)	Admin, Owner, Agent, User

الجزء ٥ — الـ (Controllers (Presentation Layer

معين feature لكيان أو endpoints كل واحد يمثل مجموعة، digital employee/Controllers، في controller ٢٩

Controller	Route	الوظيفة
AuthController	/api/Auth	تسجيل/دخول، Google OAuth، ترقية لـ Owner/Admin
BusinessController	/api/Business	onboarding flow، business، CRUD لـ
UserController	/api/User	إدارة الأدوار، agents، يوزرز، إنشاء، CRUD
CustomerController	/api/Customer	عملاء، CRUD
MenuCategoryController	/api/MenuCategory	تصنيفات المنيو، CRUD
MenuItemController	/api/MenuItem	أصناف المنيو، التوفر والسعر، CRUD
KnowledgeBaseController	/api/KnowledgeBase	(Q&A) قاعدة المعرفة، CRUD
FAQController	/api/FAQ	(KB بـ IsFAQ=true) الأسئلة الشائعة، CRUD
OrderController	/api/Order	audit logging أوردرات، تحديث الحالة مع، CRUD
TicketController	/api/Ticket	التصعيدات غير المسندة؛ تعيين وإغلاق queue تذاكر؛ CRUD
FeedbackController	/api/Feedback	(anonymous) تقييمات (يمكن، CRUD
InteractionController	/api/Interaction	Chat و Voice محادثات؛ بدء/إنهاء؛ CRUD
MessageController	/api/Message	إرسال/عرض رسائل، سجل المحادثة
SentimentController	/api/Sentiment	تحليل مشاعر رسالة
CustomerChatController	/api/CustomerChat	قدرات الشات، إرسال رسائل نصية، توصيات الأوردر
CustomerVoiceController + VoiceWebhookController	/api/CustomerVoice, /api/voice	(meetingUrl)؛ webhook لاستقبال تحليل المكالمات من الـ AI بدء مكالمات صوتية
CallRecordingController	/api/CallRecording	رفع/تحميل تسجيلات المكالمات
CallSummaryController	/api/CallSummary	استرجاع ملخص مكالمات بالـ callId أو interactionId
OwnerChatController	/api/OwnerChat	شات الـ Owner مع الـ AI لاستفسارات تحليلية
NotificationController	/api/Notification	إشعارات، تعليم كمقروء، CRUD
AuditLogController	/api/AuditLog	استعراض كل سجلات التدقيق (Admin)
DashboardController	/api/Dashboard	ملخص، تحليلات، حالة الإعداد (Owner)
AdminDashboardController	/api/AdminDashboard	تنبيهات، businesses ملخص النظام كله، أفضل (Admin)
BusinessReportController	/api/businesses/{id}/reports	توليد تقرير AI من تحليلات المحادثات لفترة زمنية
ReportController	/api/Report	تقارير محفوظة، تحميل ملفات، CRUD
SettingController	/api/Setting	قراءة/تحديث إعدادات الـ business
SubscriptionController	/api/Subscription	اشتراكات، CRUD
PaymentTransactionController	/api/PaymentTransaction	سجل عمليات الدفع
IntegrationController	/api/Integration	إدارة التكاملات الخارجية

الجزء ٦ — الـ (Service Layer) (Services)

أكثر من ٣١ service في Service layer/Services، فيهم كل الـ business logic الفعلية.

Service	الوظيفة
AuthService	تسجيل/دخول، توليد JWT (فيه BusinessId في الـ claims)، التحقق من Google ID token
BusinessService	CRUD businesses، onboarding، audit logging
UserService	ليبنس معين agents يوزرز، إنشاء CRUD
CustomerService	عملاء، بحث بالإيميل/الليبنس CRUD
MenuCategoryService	تصنيفات، ترتيب العرض CRUD
MenuItemService	عند أي تغيير AI مع الـ (sync) أصناف؛ مزامنة CRUD
KnowledgeBaseService	داخلي/FAQ ؛ تفرقة AI قاعدة المعرفة؛ مزامنة مع الـ CRUD
OrderService	الحقيقي؛ تحديث الحالة MenuItemId بالـ AI أو درات؛ ربط أسماء الأصناف اللي رجعها الـ CRUD
TicketService	لعرض التصعيدات غير المسندة؛ تعيين/إغلاق GetOpenEscalationQueueAsync تذاكر؛ CRUD
FeedbackService	Sentiment ؛ ربط بـ (1-5) rating تقييمات؛ التحقق من الـ CRUD
InteractionService	Status محادثات؛ إدارة انتقالات الـ CRUD
MessageService	ConfidenceScore، Intent و SenderType رسائل؛ تخزين CRUD
SentimentService	استدعاء الـ AI لتحليل مشاعر رسالة، أو fallback حسابي
CustomerChatService	تنسيق كامل تدفق الشات: استقبال رسالة → استدعاء الـ AI → إنشاء Order/Ticket لو محتاج → رجوع الرد
CustomerVoiceService	من الـ (HandleCallCompletedAsync للانضمام)؛ AI استدعاء + Interaction إنشاء StartCallAsync (webhook استقبال تحليل المكالمات)
AiVoiceJoinService	إرسال طلب انضمام للـ AI عند بدء مكالمة صوتية
AiChatService	استدعاء POST /api/v1/chat وتفسير رد الـ AI (نية، أورد، تذكرة، تصعيد)
AiKnowledgeSyncService	عند أي تغيير AI مع الـ FAQs لمزامنة المنيو والـ POST
AiAnalysisService	بعد انتهاء الجلسة: جلب كل الرسائل → استدعاء AI للتحليل الشامل → حفظ InteractionAnalysis
AiOwnerChatService	رد الـ AI على أسئلة الـ Owner التحليلية عن نشاطه
AiReportService	توليد تقرير AI تنفيذي، تنسيقه لعرض الـ frontend
BusinessReportGenerationService	الـ InteractionAnalysis لفترة زمنية، جميع الـ intents/topics، استدعاء AiReportService للتقرير النهائي لجميع كل
NotificationService	إنشاء إشعارات، تعليم كمقروء، دعم قنوات متعددة (Email/SMS/Push)
AuditLogService	تسجيل كل حركة مهمة (إنشاء/تعديل/حذف) بحث بالليبنس/الكيان/الحركة
SettingService	قراءة/تحديث إعدادات الـ business
ReportService	تقارير، تخزين مراجع الملفات CRUD
SubscriptionService	اشتراكات CRUD
PaymentTransactionService	عمليات دفع، ربط بالاشتراك CRUD
IntegrationService	تكاملات خارجية CRUD
DashboardService	ملخص وتحليلات لصاحب البيزنس (أوردات، عملاء جدد، تذاكر، تقييمات، مشاعر)

Service	الوظيفة
AdminDashboardService	ملخص النظام كله، أفضل businesses بالإيراد، تنبيهات، اتجاهات الإيراد

الجزء ٧ — عقود تكامل الـ (AI Integration Contracts)

فيه ٣ عقود منفصلة يتحكم التواصل بين الـ Backend والـ AI Engine، كل واحد ليه غرض مختلف تماما.

٧.١ عقد الشات (Chat Contract)

- بتتبع كل المنيو والـ — POST /api/v1/business/knowledge-base/sync (Backend → AI): مزمنة قاعدة المعرفة
- delta مش full replace بيعمل AI مع كل تغيير؛ الـ FAQs
- الرسالة نفسها، business_id، session_id (=InteractionId) — POST /api/v1/chat (Backend → AI): رسالة شات
- الرد الـ (AI → Backend): reply، order_detected/order_finalized/order_details، ticket_detected/ticket_details، escalation_requested، feedback_requested
- Escalated. الالتين (ticket_detected و escalation_requested) ممكن يكونوا true مع بعض في نفس الرد.
- من نوع Complaint؛ لو escalation_requested=true بينشئ HumanEscalation Ticket ويحدث Interaction.Status
- order_finalized=true بينشئ Order (بيدور على أسماء الأصناف في MenuItems)؛ لو ticket_detected=true بينشئ Ticket
- قرارات الـ Backend بناء على الرد: لو
- استخدامه بعد ما يتفعل (كل Interaction جديدة = session_id جديد)، والـ AI بيمسح الـ session تلقائيا بعد ساعتين من عدم النشاط.
- بتاريخ المحادثة بنفسه عن طريق الـ session_id — Backend مش محتاج بيعت الرسائل القديمة تاني. الـ session_id ميتعادش
- الـ AI بيحفظ

٧.٢ عقد التحليل (post-session) — Analysis Contract فقط

- هنا معكوس: الـ AI هو اللي بيعت للـ Backend، مش العكس. فيه Header اسمه X-AI-Contract-Version: 2.0 بيحدد نسخة العقد.
- الاتجاه
- المتحدث، اللغة/اللهجة، ولو من العميل — POST /api/conversations/sessions/{sessionId}/turns: turn لكل
- sentiment و intentDetected: الترانسكربت،
- إجمالي sentiment، summary (EN/AR) — POST /api/conversations/sessions/{sessionId}/end: نهاية الجلسة
- escalationRequired، actionsPerformed، intentsDetected، mainTopics
- Escalated ك session بيقفل الـ — POST /api/conversations/sessions/{sessionId}/escalate: تصعيد وسط الجلسة
- High بأولوية Ticket وينشئ
- CreateOrder, ModifyOrder, CancelOrder, AskAboutOrderStatus, AskAboutProducts, AskAboutPrice, AskAboutWorkingHours, AskAboutLocation, Complaint, RequestHumanAgent, Compliment, GeneralQuestion, Greeting, Farewell, Unknown: قائمة الـ Intents المدعومة:

٧.٣ عقد الصوت (Voice Contract)

- 1) POST /api/CustomerVoice/call/start → interactionId + meetingUrl استلام
- (2) العميل بيفتح الـ meetingUrl ويتكلم مع الـ AI مباشرة
- التسجيلات (POST /api/CallRecording/upload) وبيعت التحليل (POST /api/voice/call-completed — webhook)
- (3) لما المكالمة تخلص: الـ AI بيرفع
- (4) GET /api/CallSummary/{interactionId} → استرجاع الملخص الكامل + روابط الصوت

الجزء ٨ — سيناريوهات الـ Business Logic المهمة

٨.١ إنشاء أوردر من الشات

معلومة) → بينشئ Order و OrderItems جوه Unit of Work واحدة (لو أي جزء فشل، كله بيتلغي) → بيرجع orderId للعميل مع الرد. مطابق للاسم اللي اتبعت في الـ KB sync) → يحسب TotalPrice من سعر المنيو الحقيقي في قاعدة البيانات (مش من سعر الـ AI، ده مجرد مع order_details (أصناف + كميات) → الـ Backend بيدور على كل صنف بالاسم في جدول MenuItem (لازم يكون الاسم بيكتب طلبه → CustomerChatService.HandleMessageAsync ببيعته لـ AI → الـ AI بيرجع order_finalized=true للعميل

٨.٢ دورة حياة المكالمات الصوتية كاملة

Interaction إنشاء MeetingUrl، التحقق من إعداد Customer و Business، التحقق من وجود StartCallAsync: (best-effort — لو فشل، مش بيمنع الرد) AiVoiceJoinService استدعاء (channel=Voice, status=Open) •
• أثناء المكالمات: العميل بيتكلم مباشرة مع الـ AI عن طريق الـ meetingUrl (من غير ما يعدي على الـ Backend خالص)
بالـ Interaction البحث عن الـ VoiceCallCompletedDTO، استقبال HandleCallCompletedAsync (webhook): Ticket بينشئ escalationRequired=true بكل بيانات التحليل، لو CallSummary إنشاء call_id، Interaction.Status •
مش موجودة (orphan case)، الـ Backend بيسجل تحذير لكن بيكمل ويحفظ الـ CallSummary على أي حال بدل ما يرفض الطلب. ملحوظة مهمة: لو الـ Interaction

٨.٣ التصعيد و queue والتذاكر (Escalation & Ticket Queue)

بيرجع بس التذاكر GET /api/Ticket/queue → TicketService.GetOpenEscalationQueueAsync بيفتح Agent بتاعته business للـ Status=Open, TicketType=HumanEscalation, AssignedToUserId=null •
POST /api/Ticket/{id}/assign → AssignedToUserId=userId, Status=InProgress •
في نهاية مكالمات صوتية، (3) العميل بيطلب صراحة RequestHumanAgent (human agent intent). يتولد منها تصعيد: (1) الـ AI بيرجع escalation_requested=true في رد الشات، (2) الـ AI بيرجع escalationRequired=true 3 نقاط ممكن

٨.٤ الـ Feedback وربطه بالـ Sentiment

Interaction أو الـ Ticket موجود (إجباري)، وإن الـ Customer بين 1 و 5، وإن الـ Rating بيتحقق إن الـ FeedbackService SentimentScore بيتحول لـ Rating. الـ (anonymous) ممكن يتعمل من غير تسجيل دخول Feedback لو اتبعتوا) موجودين فعلا. تقريبي: 1-2 → سلبي، 3 → محايد، 4-5 → إيجابي

٨.٥ توليد تقرير الأعمال (Business Report Generation)

استثناء InteractionAnalysis موجود → بيحجب كل سجلات Business بيتحقق إن الـ BusinessReportGenerationService key والـ topics والـ intents بدل ما يرجع تقرير فاضي) → يجمع الـ NO_DATA في الفترة الزمنية المطلوبة (لو مفيش أي بيانات، بيرمي عشان يلخص كل ده في تقرير تنفيذي واحد AiReportService بيستدعي → (case-insensitive) من كل التحليلات moments

٨.٦ آلة حالات الـ Interaction (State Machine)

Open → InProgress (لما agent ينضم) | Open → Escalated (لو حصل تصعيد) | Open → Closed (من غير تصعيد)
| (بعد ما التذكرة تتحل) | InProgress → Escalated | InProgress → Closed | Escalated → Closed
للانقطاع المفاجئ Interrupted هي الحالة النهائية (مغيش انتقال بعدها). فيه كمان حالة بديلة Closed

الجزء ٩ — أسئلة متوقعة في المناقشة

ليه استخدمتوا Clean Architecture بأربع layers بدل حل أبسط؟

ال framework أو الداتا بيز ما يأتش على المنطق الأساسي. ال Service layer هو اللي بيوسط بين ال Controllers وال Domain/DAL. ال dependency خارجية (مغيش EF Core ولا ASP.NET references فيه)، فال business rules تقدر تتختبر لوحدها، وأي تغيير في عشان ال Domain layer يفضل نضيف من غير أي

إيه الفرق بين User و Customer؟

هو عميل ال Customer (JWT) و Role وعنده IdentityUser، عنده حساب حقيقي في النظام (بيمتد من admin/هو موظف User voice call أو ال chat widget عن طريق ال authentication نفسه، مش عنده حساب أو تسجيل دخول — بيتفاعل من غير business

ليه ال AI Engine سيرفر منفصل بدل ما يكون جزء من ال .NET Backend؟

(request/response). فصله بيسمح لكل سيرفر يتوسع وينشر ويفشل لوحده، ويبعد ال AI dependencies عن كود ال .NET خالص. له profile تشغيل مختلف تماما (STT/TTS، LLM inference، أحيانا محتاج GPU) مقارنة ب workload ال Backend العادي (API) ال AI Engine

مين المسؤول عن اتخاذ القرار — ال Backend ولا ال AI؟

هو اللي فعليا بينشئ ال Order أو ال Ticket في قاعدة البيانات، وبيتحقق من صحة البيانات (زي سعر المنيو الحقيقي) قبل ما يحفظها. مسار الشات. ال AI بيرجع flags واقتراحات (order_finalized, ticket_detected, escalation_requested) لكن ال Backend ال Backend دايمًا هو ال orchestrator النهائي في

إيه هو JWT Bearer Token بالظبط؟

HTTP هي الطريقة اللي بيتبعث بيها في ال Bearer، (header.payload.signature) هو شكل التوكن نفسه (نص مقسم لـ JWT userId زي claims والتوكن فيه، Program.cs في AddJwtBearer المشروع يستخدم . header: Authorization: Bearer role و BusinessId.

ليه فيه عقد منفصل لل Chat وعقد تاني لل Analysis مع إن الاتنين بيتكلموا مع نفس ال AI؟

(sentiment, topics, key moments) مش الرد الفوري. فصلهم بيمنع إبطاء رد العميل بحساب تحليل مطول مش لازم يحصل لحظيا. رد فوري محتاج يوصل للعميل). عقد ال Analysis بيتشغل بعد ما المحادثة تخلص (post-session فقط) وغرضه التحليل العميق لأن التوقيت والغرض مختلف تماما: عقد ال Chat بيتشغل live أثناء المحادثة (synchronous،

إزاي ال Backend بيتأكد إن أسماء الأصناف اللي ال AI يرجعها في الأوردر صح؟

توضيح — والسعر النهائي بيتحسب من سعر MenuItem في قاعدة البيانات مش من رقم ال AI، عشان يمنع أي تلاعب أو خطأ في السعر. (KB sync)، فال Backend بيدور على كل اسم في جدول MenuItem بمطابقة حرفية. لو الاسم مش موجود، الأوردر بيفشل أو بيتطلب ال AI بيتعامل بس مع الأسماء اللي اتبعثته في آخر (AiKnowledgeSyncService

إيه معنى إن CallSummary ممكن تتسجل ك orphan؟

ال Backend بيسجل تحذير لكن برضو بيحفظ ال CallSummary (باستخدام businessId ك fallback) بدل ما يرفض البيانات ويضيعها. وصل ومعه call_id مش متطابق مع أي Interaction موجودة (زي لو حصل خطأ في التزامن أو ال Interaction اتمسحت)، لو ال webhook بتاع call-completed

إزاي بيتحدد إن التذكرة محتاجة تصعيد بشري؟

(intent = Backend بيتشئ Ticket من نوع HumanEscalation وبيحدث حالة ال Interaction لـ Escalated. في رد الشات اللايف، escalationRequired=true في تحليل نهاية مكالمة صوتية، أو العميل يطلب صراحة (RequestHumanAgent) من ٣ مصادر ممكنة: escalation_requested=true

ليه ال Feedback ممكن يتعمل anonymous من غير Customer مسجل؟

دخول)، فالنظام يسمح بجمع تقييمات حتى لو مفيش Customer ID مربوط، طالما فيه Ticket أو Interaction مرتبطة تدي سياق للتقييم. عشان العملاء اللي بيتفاعلوا مع الشات أو الصوت أصلا مالهمش حساب (Customer entity) بسيطة، مش بتحتاج تسجيل

ليه توليد التقرير بيرمي استثناء NO_DATA بدل ما يرجع تقرير فاضي؟

اتحسب لكن طلع فاضي/صفر! لو رجعنا تقرير فاضي عادي، مش هيبقى واضح لل owner إن ده لعدم وجود بيانات أصلا وليس لخطأ في التحليل. عشان ال Frontend يقدر يفرق بوضوح بين حالتين مختلفتين: 'مفيش نشاط في الفترة دي' مقابل 'التقرير

إيه الفرق بين Ticket.Status و Ticket.PriorityLevel؟

'.يعني' فين هي دلوقتي في سير العمل — (Open/InProgress/Escalated/Closed) بيوصف مرحلة التذكرة في دورة حياتها Status في التصعيد AI بيتحدد غالبا من ثقة ال — (Low/Normal/High/Critical) PriorityLevel بيوصف مدى إلحاحها (EscalationConfidence) Status أو نوع المشكلة، ومستقل تماما عن ال

ليه فيه AuditLog منفصل بدل ما نعتد بس على تواريخ CreatedAt/UpdatedAt في كل جدول؟

إيه وامتي) بشكل مستقل عن الكيان نفسه، وده ضروري للمساءلة (accountability) خصوصا في نظام فيه أكثر من Role بيعدل نفس البيانات. لأن CreatedAt/UpdatedAt بيوريك آخر حالة بس، مش تاريخ كل التغييرات اللي حصلت. AuditLog بيسجل كل حركة لوحدها (مين عمل

إيه معنى Unit of Work عمليا في المشروع ده؟

دي مع بعض. لو حصل خطأ في نص العملية، كل حاجة بترجع (rollback)، فمفيش احتمال إن الأوردر يتسجل من غير ال OrderItems بتاعته. Order) تلمس أكثر من جدول (Order + OrderItem)، ال Unit of Work بيضمن إن SaveChanges بتحصل مرة واحدة لكل الجداول لما عملية واحدة (زي إنشاء

ليه ال KnowledgeBase Sync بيعمل full replace مش incremental update؟

مرة. ده أبسط في التنفيذ وبيمنع أي احتمال ل inconsistency بين نسخة ال AI ونسخة قاعدة البيانات الحقيقية، على حساب شوية traffic زيادة. يضمن إن ال AI دايمًا شغال بأحدث نسخة كاملة من المنيو وال FAQs من غير ما يحتاج يتتبع إيه اتغير بالظبط (add/edit/delete) في كل عشان